SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. ***Introduction:***

This section Last Minute ChatRojak is introducing the strategic solution of the identified problem statement highlighted in Product Review Documentation.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decision behind the Last Minute ChatRojak development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *A lightweight, highly efficient Node.js/Express backend integrated with an LLM service layer and a local SQLite database for rapid deployment and data persistence.*
2. ***Data Flows:***
    a. *How unstructured text from chat channels (like WhatsApp) is ingested, processed through a multi-step AI pipeline (parsing, planning, validation), and structured into actionable tasks on a user dashboard.*
3. ***Model Process:***
    a. *The end-to-end workflow of the core features: receiving text, extracting tasks, scoring them for urgency/importance, assigning dependencies, and pushing updates via the live stream UI.*
4. ***Role of Reference:***
    a. *This document serves as a high-level architectural reference for the development team to understand the AI integration, database schema, and component interactions.*

1.2. Background:

Professionals and students are increasingly overwhelmed by a constant influx of tasks across multiple chat platforms. Currently, the process of identifying, tracking, and prioritizing actionable items from unstructured conversations remains entirely manual, leading to significant inefficiency and missed deadlines

1. *Previous Version:*
    a. *The existing user workflow is fragmented across separate, non-integrated applications. It's inconvenient for users to manage tasks that originate in one platform and must be manually transferred to another. One is a "Chat Application" (like WhatsApp), and another one is a "To-Do List Application"*
2. *Changes in Major Architectural Components*
    a. *The architecture has shifted from a manual, human-driven process to an automated, AI-driven 3-Step Processing Pipeline (Extract, Plan, Validate).*
    b. *A GLM-based Extractor has been introduced to parse natural language and identify actionable tasks from raw text.*

© UMHackathon 2026

c. *A GLM-based Validator has been added as a crucial new component to fact-check the output of other AI steps, mitigating the risk of AI hallucination.*

3. *New capabilities that are only introduced in this*

a. Automatic Task Extraction : Users can now upload a WhatsApp chat file, and the system will automatically identify and create tasks, eliminating the need for manual copy-pasting.

b. Intelligent Categorization: Tasks are automatically sorted into an Eisenhower Matrix ("Do Now," "Plan" ,"Quick Win", "Later") based on AI-driven analysis of their urgency and importance.

c. Clarification Loop: The system can now identify when a task is vague (e.g., "call John about the thing") and actively prompt the user for the specific details it needs, rather than failing or creating a broken task.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Club Organizers / Businessmen	*Input unstructured chat logs, review extracted tasks, and manage project planning via the dashboard.*	*Accurate extraction of actionable items, logical prioritization, and seamless syncing to calendars*
Students	*Uploads WhatsApp chat files and manages the extracted tasks in the Power Grid UI.*	*Clear understanding of dependencies, deadlines, and deliverables without needing to read the whole chat.*
System Administrators	*Maintain the Express server, manage LLM API keys, and monitor SQLite database health.*	*Clean architecture, easy-to-read logs, and modular components for rapid debugging.*
Development Team	*Build the platform and maintain it.*	*A modular codebase with a clear separation of concerns (e.g., AI client, prompt chain, database). Well-defined error handling and retry logic to ensure system stability.*
QA Team	*Validating the system behavior for all user roles.*	*Defined scope for AI behavior, clear acceptance criteria for prompt/response pairs, and comprehensive test coverage for adversarial and edge cases.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Dashboard*
Architecture	*Client-Server architecture with integrated AI Pipeline*

ChatRojak is structured as a centralized system where inputs arrive via web form. The Node.js (Express) backend orchestrates the workflow. It passes raw data to a 3-step AI pipeline (Parser -> Planner -> Validator). The processed structured data is merged into a better-sqlite3 database and served back to the client via a web dashboard, with optional syncing to Google Calendar

2.1.2 LLM as Service Layer

The core intelligence of ChatRojak relies on a structured LLM pipeline orchestrated by promptChain.js

Step 1 (Extraction): task1Parser.js ingests raw text and normalizes it into basic tasks with deadlines and categories.

Step 2 (Planning): task2Planner.js computes urgency, importance, effort, and builds a dependency graph.

Step 3 (Validation): validator.js evaluates the output quality and triggers controlled retries if the JSON structure is malformed or lacks critical context.

2.1.3 Dependency Diagram

a. Prompts are constructed dynamically based on the step in the pipeline and sent via OpenAI/Anthropic SDKs.

b. The context window contains the user's raw text payload, the current system date/time, and the strict JSON schema expected for the output.

c. Responses are parsed natively in JavaScript. The task3Executor.js handles merging the AI output with the SQLite database, mapping IDs, and resolving conflicts.

© UMHackathon 2026

d. Input limits are managed at the entry point; excessively long chat exports are truncated or rejected before hitting the API to control token costs.

2.2.2. Sequence Diagram

i) Here, do walk through one user interaction flow or customer journey with the stakeholders [Process Chat File - Flow]

a. *User selects a .txt chat file and clicks "Upload" in the Browser.*
b. *The Browser sends a POST request containing the file's text content to the /process endpoint on the Express Server.*
c. *The Express Server receives the request and invokes the promptChain Orchestrator.*
d. *The promptChain sends the text to the AI Service (GLM/Gemini) with the Extractor prompt.*
e. *The AI Service returns a structured JSON object representing the extracted task.*
f. *The promptChain then sends this task object to the AI Service with the Categorizer prompt.*
g. *The AI Service returns the appropriate quadrant (e.g. "Do Now").*
h. *The promptChain sends the task object again to the AI Service with the Planner prompt.*
i. *The AI Service returns a list of sub-tasks and dependencies.*
j. *The promptChain performs the final call, sending all generated data to the AI Service with the Validator prompt.*
k. *The AI Service confirms the data is consistent and not a hallucination.*
l. *The promptChain saves the final, validated task object to the SQLite Database.*
m. *The promptChain checks if integrations are needed. If a due date exists, it sends the task to the Google Calendar API. Reminder before deadline notification will sends to the user by google calender.*
n. *The Express Server sends a 200 OK response back to the Browser, containing the final task data.*
o. *The Browser dynamically updates the Power Grid UI, placing the new task in its correct quadrant for the User to see.*

ii) Do demonstration of each key system features does operate from start to finish

a)AI-Powered Task Extraction: *Demonstrated during the initial extraction phase (Steps 4-5).*

b)Intelligent Prioritization: *Demonstrated when the task is assigned to a specific quadrant (Steps 6-7).*

c)Automated Task Planning: *Demonstrated when complex goals are broken down into sub-tasks (Steps 8-9).*

© UMHackathon 2026

d)Resilience and Hallucination Mitigation: *Demonstrated by the crucial Validator step which ensures data integrity (Steps 10-11).*

e)External Tool Integration: *Demonstrated by the automatic syncing with third-party calendars and messaging bots (Step 13).*

2.2. Technological Stack

2.2.1. *Frontend : Web-based UI (HTML/JS) with real-time stream listening.*
2.2.2. *Backend **:** Node.js (ESM), Express.js for REST API orchestration.*
2.2.3. *Database**:** better-sqlite3 utilizing SQL migrations for tasks, users, and dependencies.*
2.2.4. *Cloud/Deployment**:** Google Calendar API (googleCalendar.js).*

2.3. Key Data Flows

2.3.1. *Here, do show the how data are moving through the system and how its structure and stored*

2.3.1.1. Data Flow Diagram (DFD)

a.
- *Raw text moves from User -> Express Server -> AI Pipeline.*
- *Structured JSON moves from AI Pipeline -> Executor -> SQLite Database.*
- *Formatted Data moves from SQLite Database -> Dashboard / External APIs (Calendar).*

b. Handling of Improper Data (Black Hole Principle) The system is designed to be resilient and avoids the "Garbage In, Garbage Out" problem by implementing two "Black Holes" where improper data is safely discarded:

-Malformed AI Response: If the GLM service returns a conversational or malformed response instead of the required JSON, the extractJson function fails to parse it. It returns an empty object, and the promptChain immediately discards the input, preventing a system crash.
-Non-Actionable Text: If the Extractor AI correctly determines that a line of text (e.g., "hello, how are you?") is not an actionable task, it sets the isActionable flag to false. The promptChain detects this and discards the input, ensuring that only real tasks enter the system.

2.3.2. Normalized Database Schema

- *Users Table: Stores credentials and session data.*

- *Tasks Table: Stores id, title, status, urgency_score, deadline.*

- *Dependencies Table: A junction table resolving the M:N relationship between parent tasks and sub-tasks/prerequisites.-*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1.

#	Features	Description
1	AI-Powered Task Extraction	Users upload a raw chat log file. The system's Extractor AI automatically reads the conversation and extracts actionable tasks, deadlines, and details, eliminating manual data entry.
2	Intelligent Prioritization & Visualization	Extracted tasks are automatically analyzed by the Categorizer AI for urgency and importance. They are then displayed in the Power Grid UI, sorted into the four quadrants: Do Now, Plan, Quick Wins, and Later.
3	Automated Task Planning	The system's Planner AI automatically breaks down complex goals into smaller, manageable sub-tasks. Users can also define dependencies between tasks to create a clear project timeline.
4	External Tool Integration	Tasks with deadlines can be automatically synced to the user's Google Calendar.Google Calendar will send the notification reminder to the user before the deadline.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*The system must handle the processing of large chat files (up to 500kb) and multiple concurrent user sessions without crashing.*	*The Node.js server processes each request asynchronously. The AI processing for each user is a separate, non-blocking operation, allowing the server to handle multiple requests simultaneously.*
Reliability	*The AI processing pipeline should have 99% uptime, and failed API calls to the GLM must not result in lost or partially processed tasks.*	*The withRetry function in src/client.js automatically retries failed API calls to the GLM, ensuring resilience against transient network errors or service unavailability.*
Maintainability	*Each step of the AI pipeline (Extractor, Planner, Validator) is separated and can be modified individually.*	*Each AI agent's logic and prompts are encapsulated in their own module (task1Parser.js, categorizer.js, etc.). This allows a developer to improve one part of the AI chain without affecting the others.*
Token Latency	*The API calls to GLM must return a response within a reasonable time, and the user interface must not appear frozen during processing.*	*All AI API calls are asynchronous. The frontend (index.html) displays a loading spinner while waiting for the backend to process the file, providing immediate user feedback.*
Cost Efficiency	*The average cost of tokens per user session must not exceed a defined budget threshold by making unnecessary or repeated AI calls.*	*The promptChain is designed to fail fast. If the Extractor AI determines a message is not actionable, it stops processing immediately, saving the cost of the subsequent Categorizer, Planner, and Validator calls.*

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026.

a) Real-Time Chat Monitoring: Instead of uploading a static file, the system would directly integrate with chat APIs (like WhatsApp or Telegram) to monitor conversations in real-time and extract tasks as they are created.

b) Complex multi-tenant Role-Based Access Control (RBAC) for massive organizations.

c) Real-time voice note parsing (audio-to-text integration).

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

***4.1.1. Grayscale Rollout & A/B Testing**: When a new, improved prompt for the Extractor AI is developed, it is initially released to 5% of API requests. We monitor the accuracy of the extracted tasks and the rate of hallucinations. If no issues are detected, the traffic is gradually increased to 100%. After it passes, A/B testing is conducted to determine which of two prompt strategies results in lower token costs and higher accuracy.*

***4.1.2. Emergency Rollback (ER) & Golden Release**: In terms of recovery for the ChatRojak system, if there are critical anomalies after a deployment (e.g., the AI fails to return valid JSON for all requests), an ER is triggered. The automated CI/CD pipeline immediately redeploys the Golden Version of the application (the last stable commit from the main branch) to restore service instantly.*

4.1.3. Priority Matrix:

4.1.3.1. *Clearly define the Priority Matrix and what are the trigger conditions and what actions will be taken for your project.*

- *P1 Critical: If the LLM API fails or times out repeatedly, the system will return a graceful error message urging the user to try again later or input tasks manually.*

- *P2 High: If parsing succeeds but planning fails, the system bypasses the planner and inserts raw tasks into an "Unsorted/Ask User" bucket to ensure zero data loss.*

© UMHackathon 2026

4.2. Monitoring:

4.2.1. *Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.*

System health will be monitored via basic Node.js console logging, tracking API response times from OpenAI/Anthropic, and monitoring the success rate of the validator.js step.

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

a. *Users will input text that is somewhat coherent and actually contains actionable items.*

b. *Users will have their Google Calendar accounts correctly authenticated if they wish to use the scheduling features.*

4.4. External Dependencies

Tools	*Purpose*	*Risks*
ILMU GLM API/ Gemini API	*This is the core LLM reasoning engine. The system is designed to use either GLM or Gemini to handle the prompt inference, multi-step reasoning, and structured output generation for all four AI agents*	*High - API rate limits of downtime will halt the core parsing functionality.*
Google Calendar API	*Syncs scheduled tasks directly to user calendars.*	*Medium - OAuth token expiration requires user re-authentication.*

© UMHackathon 2026

5. *Project Management & Team Contributions:*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. *Project Timeline:* *Show the timeline of development and justify accordingly*

Day 1-2: *Ideation, database schema creation (SQLite), and basic Express server setup.*

Day 3-5:*Planning logic and implementation for task1Parser and task2Planner.*

Day 6-8: *Integrating frontend streams, building the dashboard UI, and refining task3Executor.*

Day 9-10: *Testing (Vitest implementation), debugging, and final presentation prep*

5.2. *Team Members Role:*

Tey Yong Zhun *- Backend & Database Architect & UI Design*
Edrian Tan Kah Heng *- Frontend & Integrations Specialist*
Teh Xu Zhe *- Pitching & Documentation*

5.3. *Recommendations:*

For future scalability, if the application traffic increases, consider migrating from better-sqlite3 to a fully hosted PostgreSQL instance, and implementing a caching layer (like Redis) for frequently accessed user dashboard states to reduce database load.

© UMHackathon 2026